МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования “Санкт-Петербургский политехнический университет Петра Великого”

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчет по лабораторной работе № 1

Операции над мультимножествами и код Грея
по дисциплине “Дискретная математика”

Выполнил:
Студент группы 5130201/50302 _____ Кузнецов М.С.

Проверил:
Преподаватель _____ Востров А.В.

“_____”_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Математическое описание **4**
1.1 Множества и мультимножества 4
1.2 Операции над мультимножествами 5
1.3 Алгоритм генерации бинарного кода Грея 6

2 Особенности реализации **8**
2.1 Структуры данных 8
2.2 Описание функций 8
2.3 Алгоритм автоматического заполнения 20
2.4 Порядок проверок при ручном заполнении 20
2.5 Защита от некорректного ввода 21

3 Результаты работы **22**
3.1 Главное меню 22
3.2 Создание универсума 22
3.3 Автоматическое заполнение 22
3.4 Ручное заполнение 23
3.5 Выполнение операций 23
3.6 Граничные случаи 24
3.7 Некорректный пункт меню 25
3.8 Некорректная разрядность 25
3.9 Мощность больше мощности универсума 26
3.10 Ошибки при ручном вводе 26
3.11 Действия не по порядку 27
3.12 Завершение ввода 27

Заключение **28**

Список литературы **30**

Приложение. Исходный код программы **31**

Введение

Цель работы – изучение мультимножеств и операций над ними, а также алгоритма генерации бинарного кода Грея; программная реализация этих понятий на языке C++.

Задание. Реализовать программу генерации бинарного кода Грея для заполнения универсума мультимножеств заданной пользователем разрядности. На основе универсума формируются два мультимножества двумя способами заполнения – вручную и автоматически (способ выбирает пользователь). Мощности множеств задаёт пользователь. В результате на экран выводятся результаты действий над множествами: объединение, пересечение, разность (оба варианта), симметрическая разность, дополнение (оба варианта), арифметические сумма, разность, произведение и деление. Кроме того, требуется реализовать защиту от некорректного пользовательского ввода и пользовательское меню.

1 Математическое описание

1.1 Множества и мультимножества

Определения множества и мультимножества приведены по учебнику [1].

Множество – это любая определённая совокупность объектов. Объекты, из которых составлено множество, называются его **элементами**; элементы множества различны и отличны друг от друга [1, с. 23]. Если объект x является элементом множества M, то говорят, что x *принадлежит* M, и пишут $x \in M$; в противном случае $x \notin M$. Множество, не содержащее элементов, называется **пустым** и обозначается $\varnothing$ [1, с. 24]. Обычно элементы всех рассматриваемых множеств берутся из некоторого одного, достаточно широкого множества U, которое называется **универсальным** множеством (или **универсумом**) [1, с. 25]. **Мощность** конечного множества $|X|$ – число его элементов [1, п. 1.2.5].

Мультимножество. В множестве все элементы различны, а значит, входят в множество ровно один раз; в мультимножестве элементы могут входить по несколько раз [1, с. 27]. Пусть $X = \{x_1, \ldots, x_n\}$ – некоторое конечное множество и $a_1, \ldots, a_n$ – неотрицательные целые числа. Тогда **мультимножеством** $\widehat{X}$ (над множеством X) называется совокупность элементов множества X, в которую элемент x_i входит a_i раз, $a_i \geq 0$. Мультимножество обозначается одним из способов [1, с. 28]:

$$\widehat{X} = [x_1^{a_1}, \ldots, x_n^{a_n}] = \langle a_1(x_1), \ldots, a_n(x_n) \rangle.$$

Например, для $X = \{a, b, c\}$ мультимножество $\widehat{X} = [a^0 b^3 c^4] = \langle b, b, b;\ c, c, c, c \rangle = \langle 0(a), 3(b), 4(c) \rangle$. Элементы мультимножества, как и элементы множества, считаются неупорядоченными.

Число a_i называется **показателем** элемента x_i, множество X – **носителем**, число $m = a_1 + \cdots + a_n$ – **мощностью** мультимножества $\widehat{X}$, а множество $\underline{X} = \{x_i \in X \mid a_i > 0\}$ – его **составом** [1, с. 28]. Для приведённого примера мощность равна $0+3+4 = 7$, а состав $\underline{X} = \{b, c\}$. Мультимножество называется **индикатором**, если $\forall i \in 1..n\ (a_i = 0 \ \vee \ a_i = 1)$ [1, с. 28].

Далее, как и в программе, мультимножества обозначаются буквами A,

B, U, а показатель элемента x в мультимножестве A называется **кратностью** и обозначается $m_A(x)$: если $A = \langle a_1(x_1), \ldots, a_n(x_n)\rangle$, то $m_A(x_i) = a_i$. Равенство $m_A(x) = 0$ означает, что x в A не входит. Мощность записывается как

$$|A| = \sum_{x \in X} m_A(x).$$

В отличие от обычного множества, мощность мультимножества не совпадает с числом элементов его состава.

В работе универсумом U называется мультимножество над множеством X кодов Грея, содержащее все допустимые элементы с максимальными кратностями. Мультимножество A является подмножеством универсума ($A \subseteq U$), если

$$m_A(x) \leq m_U(x) \quad \text{для всех } x \in X.$$

Отсюда следует, что $|A| \leq |U|$. Элементами X являются коды Грея разрядности n, а их кратности в U выбираются случайно из диапазона $[0, 10]$.

1.2 Операции над мультимножествами

Пусть A, B и универсум U – мультимножества над одним и тем же множеством X с кратностями $m_A(x)$, $m_B(x)$, $m_U(x)$ (обозначения введены в п. 1.1), причём $A \subseteq U$ и $B \subseteq U$. Операции над мультимножествами определяются поэлементно: для каждого $x \in X$ кратность $m_C(x)$ результата C вычисляется через значения $m_A(x)$, $m_B(x)$ и $m_U(x)$. Формулы, реализованные в программе, приведены в таблице 1.

Таблица 1 – Операции над мультимножествами

Операция	Обозначение	Кратность в результате
Объединение	$A \cup B$	$m_{A\cup B}(x) = \max(m_A(x),\, m_B(x))$
Пересечение	$A \cap B$	$m_{A\cap B}(x) = \min(m_A(x),\, m_B(x))$
Дополнение A	$\overline{A}$	$m_{\overline{A}}(x) = m_U(x) - m_A(x)$
Дополнение B	$\overline{B}$	$m_{\overline{B}}(x) = m_U(x) - m_B(x)$
Разность	$A \setminus B = A \cap \overline{B}$	$m_{A\setminus B}(x) = \min(m_A(x),\, m_U(x) - m_B(x))$
Разность	$B \setminus A = B \cap \overline{A}$	$m_{B\setminus A}(x) = \min(m_B(x),\, m_U(x) - m_A(x))$
Симметрическая разность	$A \triangle B = (A \setminus B) \cup (B \setminus A)$	$m_{A\triangle B}(x) = \max\big(\min(m_A(x),\, m_U(x) - m_B(x)),\, \min(m_B(x),\, m_U(x) - m_A(x))\big)$
Арифметическая сумма	$A + B$	$m_{A+B}(x) = \min(m_A(x) + m_B(x),\, m_U(x))$
Арифметическая разность	$A - B$	$m_{A-B}(x) = \max(m_A(x) - m_B(x),\, 0)$
Арифметическое произведение	$A \cdot B$	$m_{A\cdot B}(x) = \min(m_A(x) \cdot m_B(x),\, m_U(x))$
Арифметическое деление	A/B	$m_{A/B}(x) = \begin{cases} \lfloor m_A(x)/m_B(x) \rfloor, & m_B(x) > 0, \\ 0, & m_B(x) = 0 \end{cases}$

1.3 Алгоритм генерации бинарного кода Грея

Код Грея разрядности n – последовательность всех 2^n двоичных наборов длины n, в которой соседние наборы отличаются ровно в одном разряде. Код является циклическим: последний и первый наборы также отличаются в одном разряде.

Формула. i-й код Грея можно вычислить непосредственно:

$$g(i) = i \oplus (i \gg 1),$$

где $\oplus$ – побитовое исключающее ИЛИ, $\gg$ – сдвиг вправо. Именно этот способ используется в программе. Построение для $n = 3$ приведено в таблице 2.

Таблица 2 – Код Грея для $n = 3$

i	Двоичный код i	$i \gg 1$	Код Грея $g(i)$
0	000	000	000
1	001	000	001
2	010	001	011
3	011	001	010
4	100	010	110
5	101	010	111
6	110	011	101
7	111	011	100

2 Особенности реализации

2.1 Структуры данных

Универсум хранится в структуре `Universe`, содержащей два параллельных вектора: `codes` – коды Грея в виде строк и `mult` – их кратности в универсуме.

Листинг 1 – Структура Universe

```
struct Universe{
     vector<string> codes;
     vector<int> mult;
};
```

Мультимножество хранится как `vector<int>` кратностей, выровненный по индексам с универсумом: `A[i]` – кратность кода `U.codes[i]` в A. Пример приведён в таблице 3.

Таблица 3 – Хранение мультимножества ($n = 2$)

i	0	1	2	3
codes	"00"	"01"	"11"	"10"
U.mult	4	2	5	1
A	2	0	3	1

Такой вектор задаёт мультимножество $A = \{00, 00, 11, 11, 11, 10\}$, $|A| = 6$.

2.2 Описание функций

2.2.1 `int RandomInt(int min, int max)`

- **Вход:** `min`, `max` (`int`) – границы диапазона.
- **Выход:** случайное целое число из $[\texttt{min}, \texttt{max}]$.
- **Описание:** используется `uniform_int_distribution`, которое переводит значения глобального генератора `rng` (`mt19937`) в заданный диапазон с равномерным распределением.

Листинг 2 – Функция RandomInt

```
int RandomInt(int min, int max){
    uniform_int_distribution<int> dist(min, max);
    return dist(rng);
}
```

2.2.2 `vector<string> GreyGen(int n)`

- **Вход:** `n` (`int`) – разрядность кода.
- **Выход:** вектор из 2^n строк – коды Грея в порядке следования.
- **Описание:** для каждого i от 0 до $2^n - 1$ вычисляется `grey = i ^ (i >> 1)`; затем число переводится в строку из n символов: бит номер `bit` извлекается выражением `(grey >> bit) & 1`. Цикл по битам выполняется ровно n раз, поэтому ведущие нули сохраняются. Значение 2^n вычисляется как `1 << n`. При $n = 0$ функция сразу возвращает пустой вектор. Сложность $O(n \cdot 2^n)$.

Листинг 3 – Функция GreyGen

```
vector<string> GreyGen(int n){
    vector<string> codes;
    if(n == 0) return codes;
    int count = 1 << n;
    for(int i = 0; i < count; i++){
        int grey = i ^ (i >> 1);
        string code = "";
        for(int bit = n - 1; bit >= 0; bit--){
            code += ((grey >> bit) & 1) ? '1' : '0';
        }
        codes.push_back(code);
    }
    return codes;
}
```

2.2.3 `Universe GenUniverse(int n)`

- **Вход:** `n` (`int`) – разрядность кода Грея.
- **Выход:** структура `Universe`.
- **Описание:** коды получаются вызовом `GreyGen(n)`, каждому коду назначается кратность `RandomInt(0, max_mult)`, где `max_mult = 10`. Нижняя граница равна 0, поэтому часть кодов может иметь нулевую кратность, то есть фактически отсутствовать в универсуме.

Листинг 4 – Функция GenUniverse

```
Universe GenUniverse(int n){
    Universe universe;
    universe.codes = GreyGen(n);
    for(size_t i = 0; i < universe.codes.size(); i++){
        universe.mult.push_back(RandomInt(0, max_mult));
    }
    return universe;
}
```

2.2.4 `int Power(const vector<int>& mult)`

- **Вход:** `mult` – вектор кратностей мультимножества.
- **Выход:** мощность мультимножества (`int`).
- **Описание:** суммирует все кратности; нулевые кратности не влияют на результат.

Листинг 5 – Функция Power

```
int Power(const vector<int>& mult){
    int sum = 0;
    for(int m : mult){
        sum += m;
    }
    return sum;
}
```

2.2.5 `void PrintMultiset(const string& name, const vector<string>& codes, const vector<int>& mult, bool showZeros = false)`

- **Вход:** `name` – имя мультимножества; `codes` – коды универсума; `mult` – вектор кратностей; `showZeros` – выводить ли элементы с нулевой кратностью (по умолчанию нет).
- **Выход:** нет (вывод на экран).
- **Описание:** выводит мультимножество в виде `name = { <код, кратность> ... } |name| = мощность`. Элементы с нулевой кратностью печатаются только при `showZeros = true` (так выводится универсум), иначе пропускаются; пустое мультимножество выводится как `{ }`.

Листинг 6 – Функция PrintMultiset

```
void PrintMultiset(const string& name, const vector<string>& codes, const vector<int
    >& mult, bool showZeros){
    cout << name << " = { ";
    for(size_t i = 0; i < codes.size(); i++){
        if(mult[i] > 0 || showZeros){
            cout << "<" << codes[i] << ", " << mult[i] << "> ";
        }
    }
    cout << "}   |" << name << "| = " << Power(mult) << endl;
}
```

2.2.6 `int FindCode(const Universe& U, const string& code)`

- **Вход:** `U` – универсум; `code` – искомый код.
- **Выход:** индекс кода в `U.codes` или -1, если код не найден.
- **Описание:** линейный поиск, сложность $O(2^n)$.

Листинг 7 – Функция FindCode

```
int FindCode(const Universe& U, const string& code){
    for(size_t i = 0; i < U.codes.size(); i++){
        if(U.codes[i] == code){
            return i;
        }
    }
    return -1;
}
```

Общий принцип защищённого ввода: строка читается целиком и проверяется полностью. Это исключает частичное чтение, свойственное `cin >> x` (например, `12abc` было бы прочитано как 12).

2.2.7 `string ReadLine(const string& prompt)`

- **Вход:** `prompt` – текст приглашения.
- **Выход:** строка в консоли.
- **Описание:** печатает приглашение и читает строку функцией `getline`. Если ввод завершён (Ctrl+D, конец файла), выводит сообщение «Ввод прерван. Завершение программы.» и завершает программу вызовом `exit(0)`. Без этой проверки циклы повторного ввода выполнялись бы бесконечно.

Листинг 8 – Функция ReadLine

```
string ReadLine(const string& prompt){
```

```
    cout << prompt;
    string line;
    if(!getline(cin, line)){
        cout << "\nВвод прерван. Завершение программы." << endl;
        exit(0);
    }
    return line;
}
```

2.2.8 `string Trim(const string& s)`

- **Вход:** `s` – исходная строка.
- **Выход:** строка без пробелов, табуляций и символов `\r` по краям.
- **Описание:** границы значимой части находятся функциями `find_first_not_of` и `find_last_not_of`, результат выделяется `substr`. Для строки из одних пробельных символов возвращается пустая строка.

Листинг 9 – Функция Trim

```
string Trim(const string& s){
    size_t start = s.find_first_not_of(" \t\r");
    if(start == string::npos) return "";
    size_t end = s.find_last_not_of(" \t\r");
    return s.substr(start, end - start + 1);
}
```

2.2.9 `int ReadInt(const string& prompt, int min, int max)`

- **Вход:** `prompt` – приглашение; `min`, `max` – допустимый диапазон.
- **Выход:** корректное целое число из $[\texttt{min}, \texttt{max}]$.
- **Описание:** в бесконечном цикле читает строку, удаляет пробелы по краям и преобразует её функцией `stoi(line, &pos)`. Значение принимается, только если `pos == line.size()` (число занимает всю строку) и оно лежит в диапазоне. Исключения `invalid_argument` (не число) и `out_of_range` (переполнение `int`) перехватываются. При ошибке выводится сообщение с указанием диапазона и ввод повторяется.

Листинг 10 – Функция ReadInt

```
int ReadInt(const string& prompt, int min, int max){
    while(true){
```

```
        string line = Trim(ReadLine(prompt));
        try{
            size_t pos;
            int value = stoi(line, &pos);
            if(pos == line.size() && value >= min && value <= max){
                return value;
            }
        }
        catch(const invalid_argument&){ }
        catch(const out_of_range&){ }
        cout << "Ошибка: введите целое число от " << min << " до " << max << endl;
    }
}
```

2.2.10 `bool IsBinary(const string& s)`

- **Вход:** `s` – проверяемая строка.
- **Выход:** `true`, если строка непустая и состоит только из символов `0` и `1`; иначе `false`.
- **Описание:** посимвольная проверка.

Листинг 11 – Функция IsBinary

```
bool IsBinary(const string& s){
    if(s.empty()) return false;
    for(char c : s){
        if(c != '0' && c != '1') return false;
    }
    return true;
}
```

2.2.11 `vector<int> FillAuto(const Universe& U, int power)`

- **Вход:** `U` – универсум; `power` – требуемая мощность, $0 \le$ `power` $\le |U|$.
- **Выход:** вектор кратностей мультимножества мощности `power`.
- **Описание:** алгоритм «мешка»: в вектор `bag` каждый индекс `i` помещается `U.mult[i]` раз, вектор перемешивается функцией `shuffle`, из первых `power` элементов формируются кратности. Подробно алгоритм рассмотрен в п. 2.3.

Листинг 12 – Функция FillAuto

```
vector<int> FillAuto(const Universe& U, int power){
    vector<int> bag;
    for(size_t i = 0; i < U.codes.size(); i++){
        for(int j = 0; j < U.mult[i]; j++){
            bag.push_back(i);
```

```
        }
    }
    shuffle(bag.begin(), bag.end(), rng);

    vector<int> A(U.codes.size(), 0);
    for(int k = 0; k < power; k++){
        A[bag[k]]++;
    }
    return A;
}
```

2.2.12 `vector<int> FillManual(const Universe& U, int power)`

- **Вход:** `U` – универсум; `power` – требуемая мощность.
- **Выход:** вектор кратностей мультимножества мощности `power`.
- **Описание:** пока число добавленных элементов `filled` меньше `power`, пользователь вводит код и кратность; каждый ввод проходит последовательность проверок (п. 2.4). Кратность запрашивается через `ReadInt` с верхней границей `min(available, power - filled)`.

Листинг 13 – Функция FillManual

```
vector<int> FillManual(const Universe& U, int power){
    vector<int> A(U.codes.size(), 0);
    int filled = 0;
    while(filled < power){
        cout << "Осталось добавить элементов: " << power - filled << endl;
        string code = Trim(ReadLine("Введите код Грея: "));
        if(!IsBinary(code)){
            cout << "Ошибка: код должен состоять только из символов 0 и 1" << endl;
            continue;
        }
        if(code.size() != U.codes[0].size()){
            cout << "Ошибка: длина кода должна быть " << U.codes[0].size() << endl;
            continue;
        }
        int i = FindCode(U, code);
        if(i == -1){
            cout << "Ошибка: код Грея не найден в универсуме." << endl;
            continue;
        }
        if(U.mult[i] == 0){
            cout << "Ошибка: кратность этого элемента в универсуме равна 0, его
    нельзя добавить" << endl;
            continue;
        }
        int available = U.mult[i] - A[i];
        if(available == 0){
            cout << "Ошибка: элемент уже взят максимальное число раз" << endl;
            continue;
        }
        int maxCount = min(available, power - filled);
```

```
        int count = ReadInt("Кратность (1–" + to_string(maxCount) + "): ", 1,
    maxCount);
        A[i] += count;
        filled += count;
    }
    return A;
}
```

2.2.13 vector<int> CreateMultiset(const string& name, const Universe& U)

- **Вход:** `name` – имя мультимножества для вывода; `U` – универсум.
- **Выход:** вектор кратностей созданного мультимножества.
- **Описание:** запрашивает мощность в диапазоне $[0, |U|]$ и способ заполнения (1 – вручную, 2 – автоматически), вызывает `FillManual` или `FillAuto`. Если универсум пуст ($|U| = 0$, например при $n = 0$), сразу выводится сообщение «Универсум пуст» и возвращается нулевой вектор. Функция общая для A и B, что исключает дублирование кода.

Листинг 14 – Функция CreateMultiset

```
vector<int> CreateMultiset(const string& name, const Universe& U){
    int total = Power(U.mult);
    if(total == 0){
        cout << "Универсум пуст, " << name << " = ∅" << endl;
        return vector<int>(U.codes.size(), 0);
    }
    int power = ReadInt("Мощность " + name + " (0–" + to_string(total) + "): ", 0,
    total);
    int mode = ReadInt("Способ заполнения " + name + " (1 – вручную, 2 –
    автоматически): ", 1, 2);
    if(mode == 1) return FillManual(U, power);
    return FillAuto(U, power);
}
```

2.2.14 Union(A, B), Intersection(A, B)

- **Вход:** `A`, `B` – векторы кратностей.
- **Выход:** вектор кратностей $\max(a, b)$ и $\min(a, b)$ соответственно.
- **Описание:** поэлементный максимум и минимум. Функция названа `Union` с заглавной буквы, так как `union` – ключевое слово C++.

Листинг 15 – Функции Union и Intersection

```
//  max(a, b)
```

```cpp
vector<int> Union(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = max(A[i], B[i]);
    }
    return C;
}

//  min(a, b)
vector<int> Intersection(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i], B[i]);
    }
    return C;
}
```

2.2.15 `Complement(U, A)`

- **Вход:** `U` – универсум; `A` – вектор кратностей.
- **Выход:** вектор кратностей $u - a$.
- **Описание:** поэлементное вычитание из кратностей универсума.

Листинг 16 – Функция Complement

```cpp
// u - a
vector<int> Complement(const Universe& U, const vector<int>& A){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = U.mult[i] - A[i];
    }
    return C;
}
```

2.2.16 `Difference(U, A, B)`

- **Вход:** `U` – универсум; `A`, `B` – векторы кратностей.
- **Выход:** вектор кратностей $\min(a,\ u - b)$.
- **Описание:** реализована не отдельным циклом, а композицией базовых операций, прямо по определению $A \setminus B = A \cap \overline{B}$. Оба варианта разности получаются вызовами с аргументами (A, B) и (B, A).

Листинг 17 – Функция Difference

```cpp
// A \ B = A ∩ ¬B
vector<int> Difference(const Universe& U, const vector<int>& A, const vector<int>& B
    ){
    return Intersection(A, Complement(U, B));
}
```

2.2.17 `SymDifference(U, A, B)`

- **Вход:** `U` – универсум; `A`, `B` – векторы кратностей.
- **Выход:** вектор кратностей $A \triangle B$.
- **Описание:** вычисляется как `Union(Difference(U, A, B), Difference(U, B, A))`.

Листинг 18 – Функция SymDifference

```cpp
// (A \ B) ∪ (B \ A)
vector<int> SymDifference(const Universe& U, const vector<int>& A, const vector<int>& B){
    return Union(Difference(U, A, B), Difference(U, B, A));
}
```

2.2.18 `ArithSum(U, A, B)`, `ArithProduct(U, A, B)`

- **Вход:** `U` – универсум; `A`, `B` – векторы кратностей.
- **Выход:** векторы кратностей $\min(a + b, u)$ и $\min(a \cdot b, u)$.
- **Описание:** поэлементная сумма (произведение) с ограничением сверху кратностью универсума.

Листинг 19 – Функции ArithSum и ArithProduct

```cpp
// min(a + b, u)
vector<int> ArithSum(const Universe& U, const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i] + B[i], U.mult[i]);
    }
    return C;
}

// min(a * b, u)
vector<int> ArithProduct(const Universe& U, const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i] * B[i], U.mult[i]);
    }
    return C;
}
```

2.2.19 `ArithDiff(A, B)`

- **Вход:** `A`, `B` – векторы кратностей.
- **Выход:** вектор кратностей $\max(a - b, 0)$.

- **Описание:** поэлементная разность с ограничением снизу нулём.

Листинг 20 – Функция ArithDiff

```
//  max(a - b, 0)
vector<int> ArithDiff(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = max(A[i] - B[i], 0);
    }
    return C;
}
```

2.2.20 `ArithDivision(A, B)`

- **Вход:** `A`, `B` – векторы кратностей.
- **Выход:** вектор кратностей $\lfloor a/b \rfloor$; при $b = 0$ – 0.
- **Описание:** выражение `(B[i] == 0) ? 0 : A[i] / B[i]`; проверка исключает аварийное завершение при делении на ноль.

Листинг 21 – Функция ArithDivision

```
//  a / b
vector<int> ArithDivision(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = (B[i] == 0) ? 0 : A[i] / B[i];
    }
    return C;
}
```

2.2.21 `int main()`

- **Вход:** инициализация параметров программы.
- **Выход:** целое число – код завершения программы: `0`.
- **Описание:** хранит универсум `U`, мультимножества `A`, `B` и флаги их готовности. В цикле выводит меню, читает пункт через `ReadInt(0, 6)` и передаёт его в `switch`: каждая ветка проверяет флаги и вызывает нужную функцию либо выводит сообщение об ошибке. Пункт 0 завершает программу.

Листинг 22 – Функция main

```
int main(){
    Universe U;
```

```
    vector<int> A, B;
    bool hasUniverse = false;
    bool hasA = false;
    bool hasB = false;

    while(true){
        PrintMenu();
        int choice = ReadInt("Выберите пункт: ", 0, 6);

        switch(choice){
            case 1: {
                int n = ReadInt("Разрядность кода Грея (0–" + to_string(max_bits) +
"): ", 0, max_bits);
                U = GenUniverse(n);
                hasUniverse = true;
                hasA = false;
                hasB = false;
                PrintMultiset("U", U.codes, U.mult, true);
                break;
            }
            case 2:
                if(!hasUniverse){
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                PrintMultiset("U", U.codes, U.mult, true);
                break;
            case 3:
                if(!hasUniverse){
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                A = CreateMultiset("A", U);
                hasA = true;
                PrintMultiset("A", U.codes, A);
                break;
            case 4:
                if(!hasUniverse){
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                B = CreateMultiset("B", U);
                hasB = true;
                PrintMultiset("B", U.codes, B);
                break;
            case 5:
                if(!hasA && !hasB){
                    cout << "Ошибка: мультимножества ещё не заданы (пункты 3 и 4)"
<< endl;
                    break;
                }
                if(hasA) PrintMultiset("A", U.codes, A);
                else cout << "A не задано" << endl;
                if(hasB) PrintMultiset("B", U.codes, B);
                else cout << "B не задано" << endl;
                break;
            case 6:
                if(!hasA || !hasB){
                    cout << "Ошибка: сначала заполните A и B (пункты 3 и 4)" << endl
;
```

```
60                      break;
61                  }
62                  PrintAllOperations(U, A, B);
63                  break;
64              case 0:
65                  cout << "Выход из программы." << endl;
66                  return 0;
67          }
68      }
69  }
```

2.3 Алгоритм автоматического заполнения

Алгоритм `FillAuto` реализует случайную выборку без возвращения:

1. в вектор `bag` («мешок») индекс i помещается ровно $m_U(i)$ раз, всего в мешке $|U|$ элементов;
2. мешок случайно перемешивается функцией `shuffle` с генератором `rng`;
3. берутся первые `power` элементов, для каждого увеличивается соответствующая кратность `A[i]`.

Корректность. Индекс i не может быть извлечён из мешка больше раз, чем он там содержится, поэтому условие $m_A(i) \le m_U(i)$ выполняется автоматически, без дополнительных проверок. Извлекается ровно `power` элементов, поэтому $|A| =$ `power`. Сложность по времени и памяти – $O(|U|)$.

2.4 Порядок проверок при ручном заполнении

При ручном заполнении каждый введённый код проверяется в следующем порядке:

1. `IsBinary` – строка состоит только из символов `0` и `1`;
2. длина кода равна n (`U.codes[0].size()`);
3. `FindCode` $\neq -1$ – код присутствует в универсуме. Так как код Грея разрядности n перебирает все 2^n строк длины n, после проверок 1–2 код всегда найдётся; проверка сохранена как защита от выхода за границы массива;
4. $m_U(i) > 0$ – кратность элемента в универсуме ненулевая, иначе его нельзя добавить;
5. `available` $= m_U(i) - m_A(i) > 0$ – элемент ещё можно добавить;

6. кратность запрашивается через `ReadInt` в диапазоне $[1,\ \min(\texttt{available},\ \texttt{power} - \texttt{filled})]$, что не позволяет выйти ни за универсум, ни за заданную мощность.

При ошибке выводится сообщение с конкретной причиной, и оператор `continue` возвращает к повторному запросу.

2.5 Защита от некорректного ввода

Реакция программы на некорректный ввод приведена в таблице 4.

Таблица 4 – Реакция программы на некорректный ввод

Некорректный ввод	Реакция программы
`abc`, `12abc`, `1.5`, `3 7`, пустая строка	сообщение об ошибке, повторный запрос
`99999999999` (переполнение)	сообщение об ошибке, без аварийного завершения
число вне диапазона (разрядность 21, -1; мощность $> \lvert U\rvert$)	сообщение с указанием диапазона
`  5  ` с пробелами по краям	принимается как 5
код не из `0`/`1` или неверной длины	сообщение о конкретной ошибке
кратность больше доступной или больше оставшейся мощности	сообщение об ошибке, повторный запрос
пункт меню не по порядку (операции до создания A, B)	сообщение о том, что нужно сделать сначала
Ctrl+D (конец ввода)	«Ввод прерван», корректное завершение
$b = 0$ при делении	результат 0, без аварийного завершения

3 Результаты работы

Корректный ввод

3.1 Главное меню

После запуска программа выводит меню из семи пунктов и ожидает выбора (рис. 1).

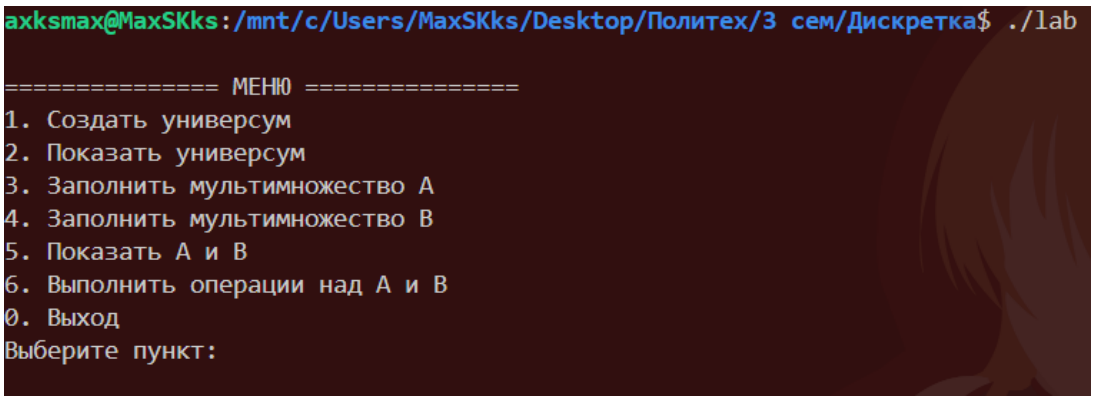


Рис. 1 – Главное меню

Меню выводится корректно.

3.2 Создание универсума

Выбирается пункт 1 и вводится разрядность 3 (допустимый диапазон – от 0 до 20). Программа должна сформировать 8 кодов Грея со случайными кратностями и вывести универсум с его мощностью (рис. 2).

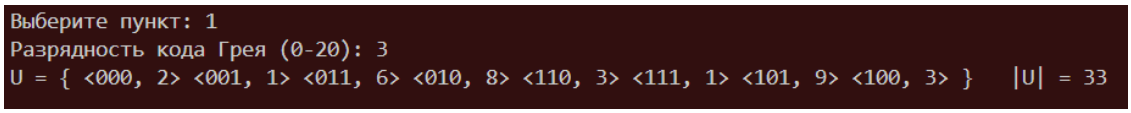


Рис. 2 – Создание универсума разрядности 3

Коды следуют в порядке кода Грея, кратности лежат в диапазоне $[0, 10]$ – универсум создан корректно.

3.3 Автоматическое заполнение

Выбирается пункт 3, задаётся мощность A, равная 5, и способ заполнения 2. Программа должна случайно сформировать A заданной

мощности (рис. 3).

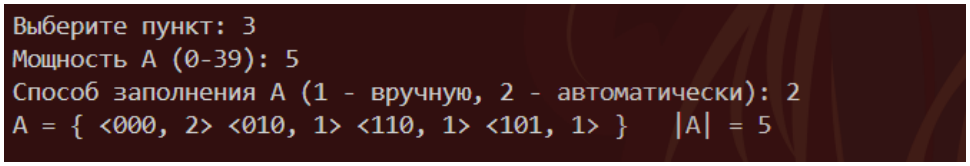


Рис. 3 – Автоматическое заполнение мультимножества A

Мощность A равна заданной, кратности не превышают кратностей универсума.

3.4 Ручное заполнение

Выбирается пункт 4, задаётся мощность B, равная 5, и способ заполнения 1; вводятся коды `111`, `010`, `100` с кратностями 2, 1, 2. Программа должна после каждого шага сообщать, сколько элементов осталось добавить (рис. 4).

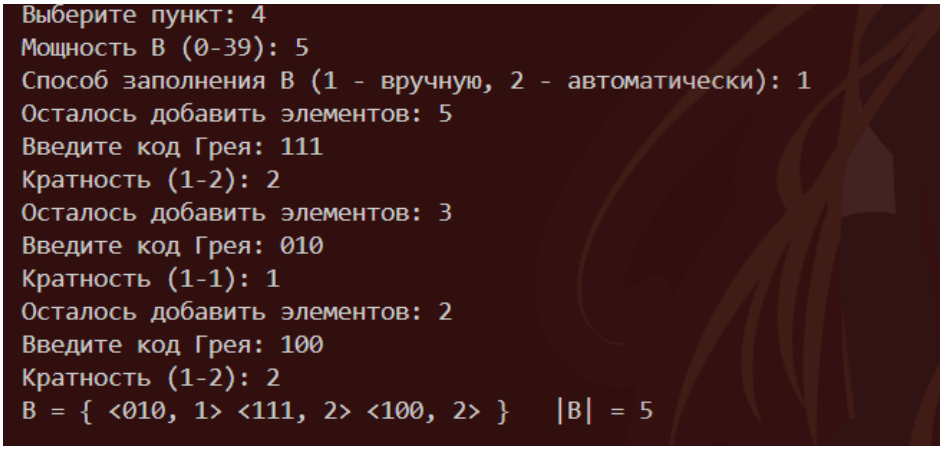


Рис. 4 – Ручное заполнение мультимножества B

Мультимножество B сформировано в точном соответствии с введёнными данными.

3.5 Выполнение операций

Выбирается пункт 6. Программа должна вывести результаты всех операций над A и B (рис. 5).

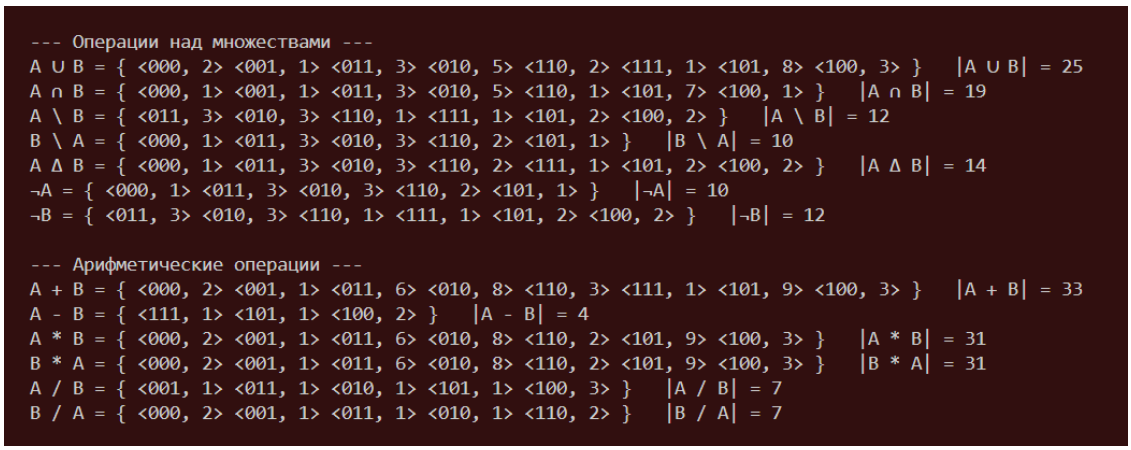


Рис. 5 – Результаты операций над A и B

Результаты соответствуют формулам таблицы 1, кратность каждого элемента лежит в пределах от 0 до $m_U(x)$.

3.6 Граничные случаи

Мультимножество A создаётся с мощностью 0, а B – с мощностью, равной $|U| = 39$; затем пункты 2 и 5 выводят универсум и оба мультимножества. Ожидается пустое A и B, совпадающее с универсумом (рис. 6).

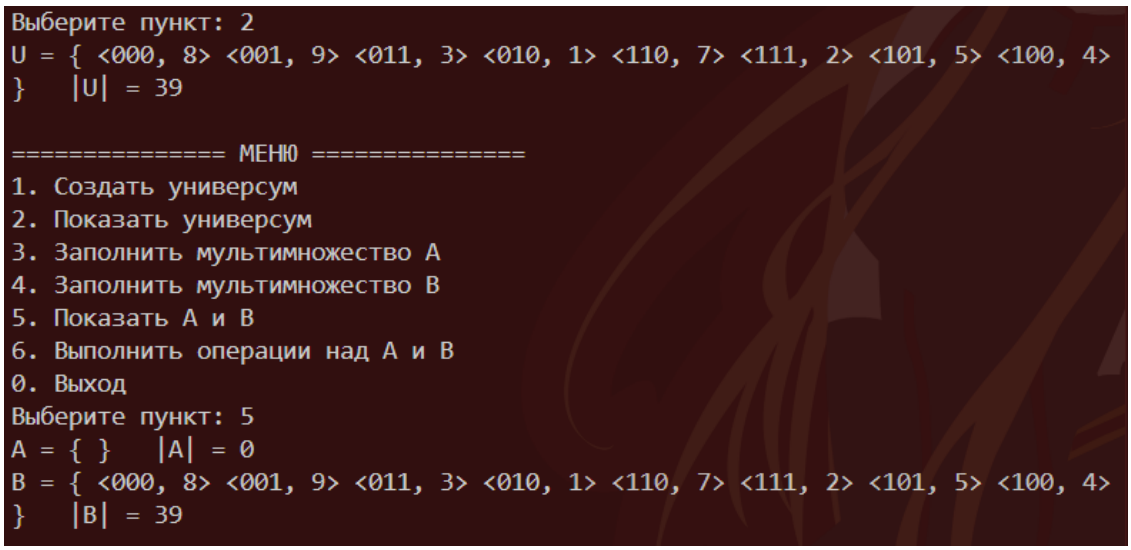


Рис. 6 – Пустое мультимножество и мультимножество мощности $|U|$

Пустое мультимножество выводится как `{ }`, а при мощности $|U|$ результат совпадает с универсумом.

Некорректный ввод

3.7 Некорректный пункт меню

В меню вводятся `abc`, `7`, `-1` и пустая строка. Каждый ввод должен быть отклонён (рис. 7).

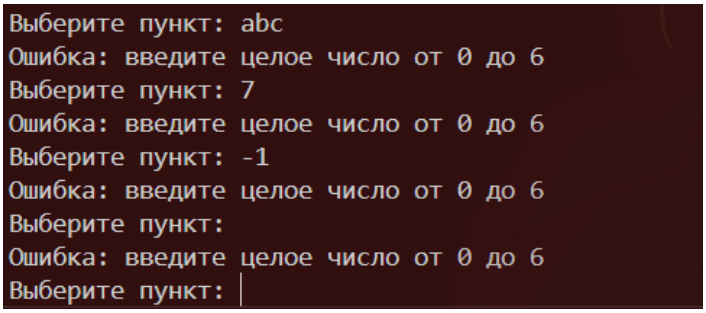


Рис. 7 – Некорректный ввод в меню

На каждый ввод выведено сообщение «Ошибка: введите целое число от 0 до 6», программа продолжила работу.

3.8 Некорректная разрядность

При создании универсума вводятся `0`, `11`, `12abc` и `a12`. Все значения должны быть отклонены (рис. 8).

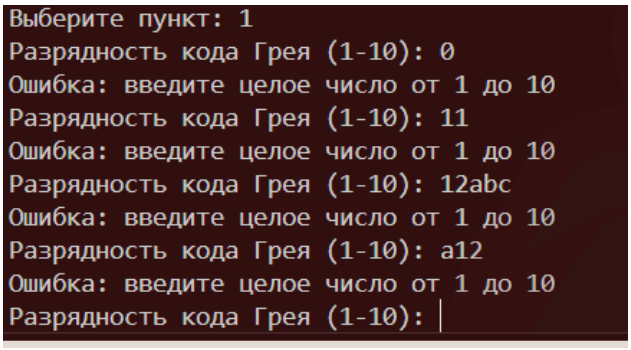


Рис. 8 – Некорректная разрядность кода Грея

Значения вне диапазона и строка с посторонними символами отклонены, повторный запрос выполнен.

3.9 Мощность больше мощности универсума

При заполнении A вводится мощность 28 при $|U| = 27$. Ввод должен быть отклонён (рис. 9).

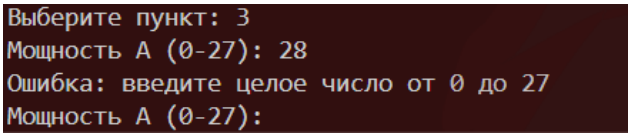


Рис. 9 – Мощность больше $|U|$

Программа сообщила допустимый диапазон $[0, |U|]$ и повторила запрос.

3.10 Ошибки при ручном вводе

При ручном заполнении ($n = 2$) вводятся код `012`, код неверной длины `000`, кратность 6 при допустимом максимуме 4 и код `01`, уже взятый максимальное число раз (рис. 10).

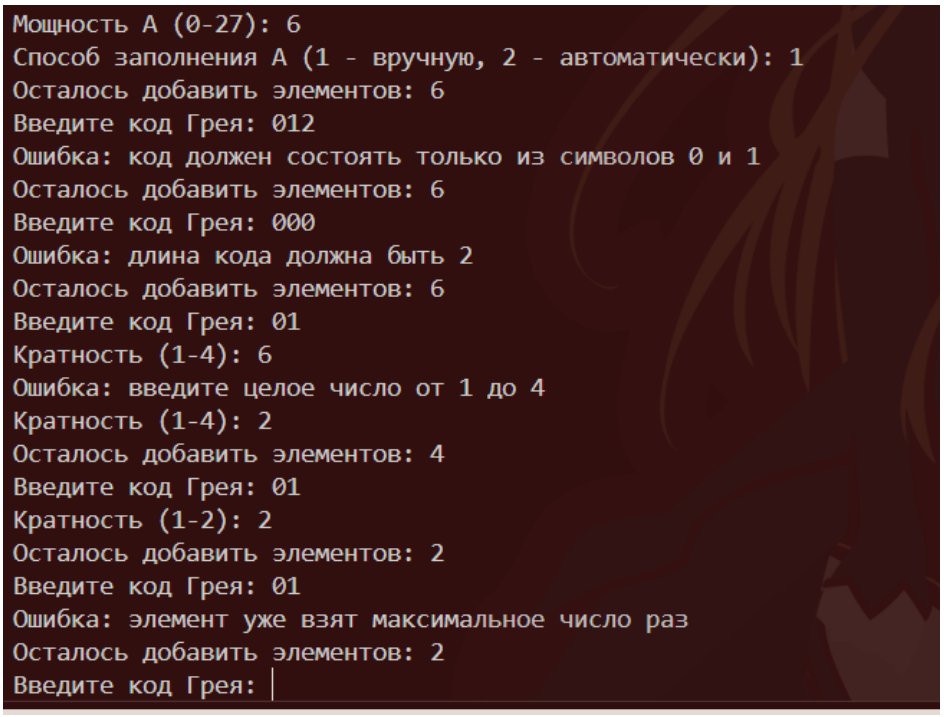


Рис. 10 – Ошибки при ручном заполнении

Для каждой ошибки выведено сообщение с её конкретной причиной, мультимножество не вышло за пределы универсума.

3.11 Действия не по порядку

До создания универсума выбирается пункт 3, а до заполнения A и B – пункт 6. Оба действия должны быть отклонены (рис. 11).

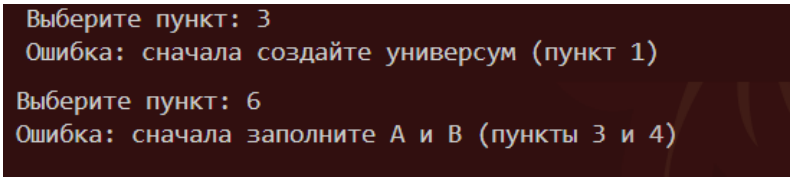


Рис. 11 – Действия в неверном порядке

Программа сообщает, какие пункты нужно выполнить сначала, и возвращается в меню.

3.12 Завершение ввода

В меню нажимается Ctrl+D (конец ввода). Программа должна корректно завершиться (рис. 12).

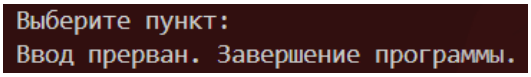


Рис. 12 – Завершение ввода (Ctrl+D)

Выведено сообщение «Ввод прерван. Завершение программы.», программа завершилась без зацикливания.

Заключение

В ходе работы реализована программа на C++, которая:

- генерирует бинарный код Грея заданной разрядности $n \in [0, 20]$ по формуле $g(i) = i \oplus (i \gg 1)$;
- формирует универсум из 2^n кодов Грея со случайными кратностями из $[0, 10]$;
- формирует мультимножества A и B заданной мощности вручную или автоматически;
- вычисляет и выводит результаты операций: объединение, пересечение, разность (оба варианта), симметрическую разность, дополнение (оба варианта), арифметические сумму, разность, произведение и деление;
- защищена от некорректного ввода и управляется через меню.

Достоинства:

- мультимножество хранится как вектор кратностей, выровненный по индексам с универсумом, поэтому каждая операция выполняется одним проходом за $O(2^n)$;
- разность и симметрическая разность выражены через базовые операции, код совпадает с математическими определениями;
- автоматическое заполнение гарантированно не выходит за пределы универсума.

Недостатки:

- поиск кода при ручном вводе линейный, $O(2^n)$;
- автоматическое заполнение требует $O(|U|)$ дополнительной памяти на «мешок».

Масштабирование. На основе уже реализованных функций программу можно дополнить следующими возможностями с минимальными изменениями кода.

- **Использование результата операции как нового операнда:** пункт меню, который сохраняет результат выбранной операции в A или B (присваиванием вектора, возвращённого функцией операции), позволяет строить цепочки вычислений без повторного ввода данных.
- **Проверка отношений между мультимножествами:** включение $A \subseteq B$ ($m_A(x) \le m_B(x)$ для всех x) и равенство ($m_A(x) = m_B(x)$ для всех x)

– одной функцией, возвращающей `bool`, за один проход по кратностям, по той же схеме, что и операции (сложность $O(2^n)$).

- **Операции над несколькими мультимножествами**, например $A \cap B \cap C$ или $(A \cup B) \setminus C$: функции операций возвращают новый вектор кратностей, поэтому их можно применять последовательно к результатам друг друга.
- **Арифметическая степень** A^k с кратностями $\min(m_A(x)^k,\, m_U(x))$ – повторным применением `ArithProduct`, и **умножение на число** $k \cdot A$ с кратностями $\min(k \cdot m_A(x),\, m_U(x))$ – повторным применением `ArithSum`.
- **Ввод и вывод мультимножеств в файл.** Функцию `PrintMultiset` достаточно направить в файловый поток вместо экрана (заменить `cout` параметром типа `ostream&`), а ввод – реализовать чтением пар «код; кратность» из файла с той же проверкой диапазона, что и в `ReadInt`.

Список литературы

1. Новиков Ф. А. Дискретная математика для программистов: учебник для вузов. – 3-е изд. – СПб.: Питер, 2009. – 384 с.
2. cppreference.com: справочник по языку C++ и стандартной библиотеке [Электронный ресурс]. – URL: `https://en.cppreference.com` (дата обращения: 27.09.2026).

Приложение. Исходный код программы

Листинг 23 – main.cpp

```
#include <iostream>
#include <vector>
#include "input.h"
#include "universe.h"
#include "multiset.h"
#include "operations.h"
using namespace std;


const int max_bits = 20;

void PrintMenu(){
    cout << "\n=============== МЕНЮ ===============" << endl;
    cout << "1. Создать универсум" << endl;
    cout << "2. Показать универсум" << endl;
    cout << "3. Заполнить мультимножество A" << endl;
    cout << "4. Заполнить мультимножество B" << endl;
    cout << "5. Показать A и B" << endl;
    cout << "6. Выполнить операции над A и B" << endl;
    cout << "0. Выход" << endl;
}

int main(){
    Universe U;
    vector<int> A, B;
    bool hasUniverse = false;
    bool hasA = false;
    bool hasB = false;

    while(true){
        PrintMenu();
        int choice = ReadInt("Выберите пункт: ", 0, 6);

        switch(choice){
            case 1: {
                int n = ReadInt("Разрядность кода Грея (0–" + to_string(max_bits) +
    "): ", 0, max_bits);
                U = GenUniverse(n);
                hasUniverse = true;
                hasA = false;
                hasB = false;
                PrintMultiset("U", U.codes, U.mult, true);
                break;
            }
            case 2:
                if(!hasUniverse){
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                PrintMultiset("U", U.codes, U.mult, true);
                break;
            case 3:
                if(!hasUniverse){
```

```
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                A = CreateMultiset("A", U);
                hasA = true;
                PrintMultiset("A", U.codes, A);
                break;
            case 4:
                if(!hasUniverse){
                    cout << "Ошибка: сначала создайте универсум (пункт 1)" << endl;
                    break;
                }
                B = CreateMultiset("B", U);
                hasB = true;
                PrintMultiset("B", U.codes, B);
                break;
            case 5:
                if(!hasA && !hasB){
                    cout << "Ошибка: мультимножества ещё не заданы (пункты 3 и 4)"
    << endl;
                    break;
                }
                if(hasA) PrintMultiset("A", U.codes, A);
                else cout << "A не задано" << endl;
                if(hasB) PrintMultiset("B", U.codes, B);
                else cout << "B не задано" << endl;
                break;
            case 6:
                if(!hasA || !hasB){
                    cout << "Ошибка: сначала заполните A и B (пункты 3 и 4)" << endl
    ;
                    break;
                }
                PrintAllOperations(U, A, B);
                break;
            case 0:
                cout << "Выход из программы." << endl;
                return 0;
        }
    }
}
```

Листинг 24 – input.h

```
#pragma once
#include <string>
using namespace std;

string ReadLine(const  string& prompt);

string Trim(const  string& s);

int ReadInt(const  string& prompt, int min, int max);

bool IsBinary(const  string& s);
```

Листинг 25 – input.cpp

```cpp
#include "input.h"
#include <iostream>
#include <cstdlib>
#include <stdexcept>
using namespace std;

string ReadLine(const string& prompt){
    cout << prompt;
    string line;
    if(!getline(cin, line)){
        cout << "\nВвод прерван. Завершение программы." << endl;
        exit(0);
    }
    return line;
}

string Trim(const string& s){
    size_t start = s.find_first_not_of(" \t\r");
    if(start == string::npos) return "";
    size_t end = s.find_last_not_of(" \t\r");
    return s.substr(start, end - start + 1);
}

int ReadInt(const string& prompt, int min, int max){
    while(true){
        string line = Trim(ReadLine(prompt));
        try{
            size_t pos;
            int value = stoi(line, &pos);
            if(pos == line.size() && value >= min && value <= max){
                return value;
            }
        }
        catch(const invalid_argument&){ }
        catch(const out_of_range&){ }
        cout << "Ошибка: введите целое число от " << min << " до " << max << endl;
    }
}

bool IsBinary(const string& s){
    if(s.empty()) return false;
    for(char c : s){
        if(c != '0' && c != '1') return false;
    }
    return true;
}
```

Листинг 26 – universe.h

```cpp
#pragma once
#include <string>
#include <vector>
#include <random>
using namespace std;

extern  mt19937 rng;


struct Universe{
```

```
     vector<string> codes;
     vector<int> mult;
};


int RandomInt(int min, int max);

vector< string> GreyGen(int n);

Universe GenUniverse(int n);

int Power(const  vector<int>& mult);

void PrintMultiset(const  string& name, const  vector< string>& codes, const  vector
    <int>& mult, bool showZeros = false);

int FindCode(const Universe& U, const  string& code);
```

Листинг 27 – universe.cpp

```
#include "universe.h"
#include <iostream>
using namespace std;

const int max_mult = 10;
mt19937 rng(random_device{}());

int RandomInt(int min, int max){
    uniform_int_distribution<int> dist(min, max);
    return dist(rng);
}

vector<string> GreyGen(int n){
    vector<string> codes;
    if(n == 0) return codes;
    int count = 1 << n;
    for(int i = 0; i < count; i++){
        int grey = i ^ (i >> 1);
        string code = "";
        for(int bit = n - 1; bit >= 0; bit--){
            code += ((grey >> bit) & 1) ? '1' : '0';
        }
        codes.push_back(code);
    }
    return codes;
}

Universe GenUniverse(int n){
    Universe universe;
    universe.codes = GreyGen(n);
    for(size_t i = 0; i < universe.codes.size(); i++){
        universe.mult.push_back(RandomInt(0, max_mult));
    }
    return universe;
}

int Power(const vector<int>& mult){
    int sum = 0;
    for(int m : mult){
```

```
        sum += m;
    }
    return sum;
}

void PrintMultiset(const string& name, const vector<string>& codes, const vector<int
    >& mult, bool showZeros){
    cout << name << " = { ";
    for(size_t i = 0; i < codes.size(); i++){
        if(mult[i] > 0 || showZeros){
            cout << "<" << codes[i] << ", " << mult[i] << "> ";
        }
    }
    cout << "}   |" << name << "| = " << Power(mult) << endl;
}

int FindCode(const Universe& U, const string& code){
    for(size_t i = 0; i < U.codes.size(); i++){
        if(U.codes[i] == code){
            return i;
        }
    }
    return -1;
}
```

Листинг 28 – multiset.h

```
#pragma once
#include <string>
#include <vector>
#include "universe.h"
using namespace std;

 vector<int> FillAuto(const Universe& U, int power);


 vector<int> FillManual(const Universe& U, int power);

 vector<int> CreateMultiset(const  string& name, const Universe& U);
```

Листинг 29 – multiset.cpp

```
#include "multiset.h"
#include "input.h"
#include <iostream>
#include <algorithm>
using namespace std;

vector<int> FillAuto(const Universe& U, int power){
    vector<int> bag;
    for(size_t i = 0; i < U.codes.size(); i++){
        for(int j = 0; j < U.mult[i]; j++){
            bag.push_back(i);
        }
    }
    shuffle(bag.begin(), bag.end(), rng);

```

```
    vector<int> A(U.codes.size(), 0);
    for(int k = 0; k < power; k++){
        A[bag[k]]++;
    }
    return A;
}

vector<int> FillManual(const Universe& U, int power){
    vector<int> A(U.codes.size(), 0);
    int filled = 0;
    while(filled < power){
        cout << "Осталось добавить элементов: " << power - filled << endl;
        string code = Trim(ReadLine("Введите код Грея: "));
        if(!IsBinary(code)){
            cout << "Ошибка: код должен состоять только из символов 0 и 1" << endl;
            continue;
        }
        if(code.size() != U.codes[0].size()){
            cout << "Ошибка: длина кода должна быть " << U.codes[0].size() << endl;
            continue;
        }
        int i = FindCode(U, code);
        if(i == -1){
            cout << "Ошибка: код Грея не найден в универсуме." << endl;
            continue;
        }
        if(U.mult[i] == 0){
            cout << "Ошибка: кратность этого элемента в универсуме равна 0, его
    нельзя добавить" << endl;
            continue;
        }
        int available = U.mult[i] - A[i];
        if(available == 0){
            cout << "Ошибка: элемент уже взят максимальное число раз" << endl;
            continue;
        }
        int maxCount = min(available, power - filled);
        int count = ReadInt("Кратность (1-" + to_string(maxCount) + "): ", 1,
    maxCount);
        A[i] += count;
        filled += count;
    }
    return A;
}

vector<int> CreateMultiset(const string& name, const Universe& U){
    int total = Power(U.mult);
    if(total == 0){
        cout << "Универсум пуст, " << name << " = ∅" << endl;
        return vector<int>(U.codes.size(), 0);
    }
    int power = ReadInt("Мощность " + name + " (0-" + to_string(total) + "): ", 0,
    total);
    int mode = ReadInt("Способ заполнения " + name + " (1 - вручную, 2 -
    автоматически): ", 1, 2);
    if(mode == 1) return FillManual(U, power);
    return FillAuto(U, power);
}
```

Листинг 30 – operations.h

```h
#pragma once
#include <vector>
#include "universe.h"
using namespace std;

 vector<int> Union(const  vector<int>& A, const  vector<int>& B);
 vector<int> Intersection(const  vector<int>& A, const  vector<int>& B);
 vector<int> Complement(const Universe& U, const  vector<int>& A);
 vector<int> Difference(const Universe& U, const  vector<int>& A, const  vector<int>& B);
 vector<int> SymDifference(const Universe& U, const  vector<int>& A, const  vector<int>& B);


 vector<int> ArithSum(const Universe& U, const  vector<int>& A, const  vector<int>& B);
 vector<int> ArithDiff(const  vector<int>& A, const  vector<int>& B);
 vector<int> ArithProduct(const Universe& U, const  vector<int>& A, const  vector<int>& B);
 vector<int> ArithDivision(const  vector<int>& A, const  vector<int>& B);


void PrintAllOperations(const Universe& U, const  vector<int>& A, const  vector<int>& B);
```

Листинг 31 – operations.cpp

```cpp
#include "operations.h"
#include <iostream>
#include <algorithm>
using namespace std;

//  max(a, b)
vector<int> Union(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = max(A[i], B[i]);
    }
    return C;
}

//  min(a, b)
vector<int> Intersection(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i], B[i]);
    }
    return C;
}

// u - a
vector<int> Complement(const Universe& U, const vector<int>& A){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = U.mult[i] - A[i];
    }
    return C;
```

```
}

// A \ B = A ∩ ¬B
vector<int> Difference(const Universe& U, const vector<int>& A, const vector<int>& B
    ){
    return Intersection(A, Complement(U, B));
} //min(a, u - b)

// (A \ B) ∪ (B \ A)
vector<int> SymDifference(const Universe& U, const vector<int>& A, const vector<int
    >& B){
    return Union(Difference(U, A, B), Difference(U, B, A));
}

// min(a + b, u)
vector<int> ArithSum(const Universe& U, const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i] + B[i], U.mult[i]);
    }
    return C;
}

//  max(a - b, 0)
vector<int> ArithDiff(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = max(A[i] - B[i], 0);
    }
    return C;
}

// min(a * b, u)
vector<int> ArithProduct(const Universe& U, const vector<int>& A, const vector<int>&
     B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = min(A[i] * B[i], U.mult[i]);
    }
    return C;
}

//  a / b
vector<int> ArithDivision(const vector<int>& A, const vector<int>& B){
    vector<int> C(A.size());
    for(size_t i = 0; i < A.size(); i++){
        C[i] = (B[i] == 0) ? 0 : A[i] / B[i];
    }
    return C;
}

void PrintAllOperations(const Universe& U, const vector<int>& A, const vector<int>&
    B){
    cout << "\n--- Операции над множествами ---" << endl;
    PrintMultiset("A ∪ B",  U.codes, Union(A, B));
    PrintMultiset("A ∩ B",  U.codes, Intersection(A, B));
    PrintMultiset("A \\ B", U.codes, Difference(U, A, B));
    PrintMultiset("B \\ A", U.codes, Difference(U, B, A));
    PrintMultiset("A Δ B",  U.codes, SymDifference(U, A, B));
    PrintMultiset("¬A",     U.codes, Complement(U, A));
```

```
87      PrintMultiset("¬B",     U.codes, Complement(U, B));
88
89      cout << "\n--- Арифметические операции ---" << endl;
90      PrintMultiset("A + B",  U.codes, ArithSum(U, A, B));
91      PrintMultiset("A − B",  U.codes, ArithDiff(A, B));
92      PrintMultiset("A * B",  U.codes, ArithProduct(U, A, B));
93      PrintMultiset("B * A",  U.codes, ArithProduct(U, B, A));
94      PrintMultiset("A / B",  U.codes, ArithDivision(A, B));
95      PrintMultiset("B / A",  U.codes, ArithDivision(B, A));
96  }
```